

HOWARD Help Calculations

1 Calculations

List of available calculations

1.1 ANNOTATION

Annotation of variants based on specified databases and tools

Annotate variants based on specified databases and tools. This calculation allows for the annotation of variants using parameters specified in the JSON parameter file in the annotation parameters (see help.annotation.pdf), including options on tools to use (HOWARD parquet algorithm, Annovar, snpEff, BCFTools...). Example that annotate variants with database in parquet format, annovar, snpEff and BCFTools with databases in VCF or BED format:

```
{
  "annotation": {
    "parquet": {
      "annotations": {
        "my.database.parquet": {...},
        "my.other.database.parquet": {...}
      }
    },
    "annovar": {...}
    "snpeff": {...}
    "bcftools": {...}
  }
}
```

1.2 BARCODE

BARCODE as VaRank tool

BARCODE as VaRank tool. This calculation generates a BARCODE for each sample based on sample genotype (0 for reference, 1 for heterozygous, 2 for homozygous alternate).

1.3 BARCODEFAMILY

BARCODEFAMILY as VaRank tool

BARCODEFAMILY as VaRank tool. This calculation generates a BARCODE for each family based on sample genotype (0 for reference, 1 for heterozygous, 2 for homozygous alternate) and family relationships.

1.4 COUNT_SAMPLES

Number of sample that have a genotype for the variant (for multi sample VCF)

Number of sample that have a genotype for the variant (for multi sample VCF). This calculation counts the number of samples (fixed 'count_samples' field) that have a genotype for a given variant in a multi-sample VCF file. It helps in assessing the presence of the variant across different samples.

1.5 DP_STATS

Depth (DP) statistics

Depth (DP) statistics. This calculation provides statistical measures for the sequencing depth (DP) across different samples. It helps in understanding the coverage and reliability of variant calls.

1.6 FIND_SAMPLES

Number of sample that have a genotype for the variant (for multi sample VCF)

This calculation counts the number of samples that have a genotype for a given variant in a multi-sample VCF file. It helps in assessing the presence of the variant across different samples.

Options for this calculation can be specified in the JSON parameter file, directly in the calculation parameters (see [help.parameters.pdf](#)). The parameter 'tags' allows for the specification of the INFO tags to be created, with the key being the tag name and the value being either 'count' or 'list' to indicate whether to count the samples or list them. For example, to create an INFO tag 'count_samples' that counts the number of samples and an INFO tag 'list_samples' that lists the samples, you can specify:

```
{ "tags": { "count_samples": "count", "list_samples": "list" } }
```

Otherwise, change the default tags in the parameter file, like in this example:

```
{ "tags": { "count_samples_for_variants": "count", "list_samples_for_variants": "list", "another_list_samples_for_variants": "list" } }
```

1.7 GENOTYPECONCORDANCE

Concordance of genotype for multi caller VCF

Concordance of genotype for multi caller VCF. This calculation assesses the concordance of genotypes for a given variant across multiple callers in a multi-caller VCF file. It helps in evaluating the consistency of genotype calls and can be used to identify variants with high confidence based on agreement among different callers.

1.8 GENOTYPE_STATS

Calculate genotype statistics on a specific genotype field (e.g. VAF, DP, GQ...)

Calculate genotype statistics on a specific genotype field (e.g. VAF, DP, GQ...). This calculation computes statistical measures for a specified genotype field across different samples, providing insights into the distribution and variability of the chosen genotype metric.

Options for this calculation can be specified in the JSON parameter file, directly in the calculation parameters (see [help.parameters.pdf](#)). The parameter 'info' allows for the specification of the genotype field to be analyzed. For example, to calculate statistics for the GQ field, you can specify:

```
{ "info": "GQ" }
```

If no 'info' parameter is provided, the calculation will default to analyzing the VAF field. Other genotype fields can be specified as needed, such as DP (Depth), GQ (Genotype Quality), etc., by providing the appropriate field name in the 'info' parameter.

1.9 INFO_TO_FORMAT

INFO to FORMAT conversion

INFO to FORMAT conversion. This calculation converts INFO fields to FORMAT fields in the VCF file.

Options for this calculation can be specified in the JSON parameter file, directly in the calculation parameters (see [help.parameters.pdf](#)):

- 'annotation_fields': allows for the specification of the INFO fields to be converted to FORMAT fields, with the key being the INFO field name and the value being the FORMAT field name. For example, to convert INFO field 'count_samples' to FORMAT field 'CS', INFO field 'list_samples' to FORMAT field 'LS', and INFO field 'calling_quality' to FORMAT field 'CQ', you can specify:

- 'samples': allows for the specification of the samples to be included in the conversion. If not specified, all samples will be included.
- 'remove_info_fields': allows for the removal of the original INFO fields after conversion to FORMAT fields. If set to true, the original INFO fields will be removed from the VCF file.

Options example:

```
{
  "annotation_fields": {
    "count_samples": null,
    "list_samples": "LS",
    "calling_quality": "CQ"
  },
  "samples": ["sample1", "sample2"],
  "remove_info_fields": true
}
```

1.10 LIST_SAMPLES

List of samples that have a genotype for the variant (for multi sample VCF)

List of samples that have a genotype for the variant (for multi sample VCF). This calculation provides a list of samples (fixed 'list_samples' field) that have a genotype for a given variant in a multi-sample VCF file. It helps in understanding which samples support the presence of the variant.

1.11 MERGED_HGVS

Merge HGVS nomenclatures from snpEff (snpeff_hgvs) and ANNOVAR (AAChange_refGene) into merged_hgvs field

This calculation merges HGVS nomenclatures from snpEff (snpeff_hgvs) and ANNOVAR (AAChange_refGene) into a single field called merged_hgvs. It aggregates distinct HGVS annotations from both sources, concatenating them with a comma separator. This unified field allows for easier access to HGVS information regardless of the annotation source, facilitating downstream analysis and interpretation.

1.12 NOMEN

NOMEN information (e.g. NOMEN, CNOMEN, PNOMEN...) from HGVS nomenclature field (see parameters help)

Extract NOMEN information (e.g. NOMEN, CNOMEN, PNOMEN...) from HGVS nomenclature field (see parameters help). This calculation parses the HGVS nomenclature field to extract specific NOMEN information such as NOMEN, CNOMEN, PNOMEN, etc., based on the parameters provided. It creates new INFO fields with the extracted NOMEN information for easier access and downstream analysis.

1.13 NOMEN_SNP EFF

NOMEN information (e.g. NOMEN, CNOMEN, PNOMEN...) from HGVS nomenclature field (see parameters help)

Extract NOMEN information (e.g. NOMEN, CNOMEN, PNOMEN...) from HGVS nomenclature field (see parameters help). This calculation parses the HGVS nomenclature field to extract specific NOMEN information such as NOMEN, CNOMEN, PNOMEN, etc., specifically based on snpEff HGVS annotations field 'snpeff_hgvs'. It creates new INFO fields with the extracted NOMEN information for easier access and downstream analysis.

1.14 PRIORITIZATION

Prioritization of variants based on specified profiles

Prioritize variants based on specified profiles. This calculation allows for the prioritization of variants using parameters specified in the JSON parameter file in the prioritization parameters (see help.prioritization.pdf), including options on profiles to use and scoring methods. Example that prioritize variants using default and GERMLINE profiles:

```
{
  "prioritization": {
    "profiles": ["default", "GERMLINE"],
```

```

    "pzfields": ["PZFlag", "PZScore", "PZClass", "PZComment", "PZInfos", "PZTags"]
  }
}

```

1.15 RECREATE_INFO_FIELDS

Recreate INFO_tags, rename or remove tags

Recreate INFO_tags, rename or remove tags. This calculation allows for the recreation of INFO fields by renaming existing tags or removing them based on specified parameters. It provides flexibility in managing INFO fields, enabling users to customize their VCF annotations according to their analysis needs.

1.16 RENAME_INFO_FIELDS

Rename or remove INFO/tags

Rename or remove INFO/tags. This calculation allows for the renaming of existing INFO fields or the removal of specific tags based on provided parameters. It helps in standardizing or customizing INFO fields in the VCF according to user requirements and facilitates downstream analysis. Use paramter section 'calculation_RENAME_INFO_FIELDS' in param.json to specify the INFO fields to rename or remove.

This example will rename INFO field 'ENOMEN' to 'exon' and remove INFO fields 'SPiP', 'SPiP_Alt' and 'SPiP_distSS' from the 'variants' table:

```

{"fields_to_rename": {"ENOMEN": "exon", "SPiP": null, "SPiP_Alt": null, "SPiP_distSS": null }, "table": "vari

```

1.17 SNPEFF__ANN__EXPLODE

Explode snpEff annotations

Explode snpEff annotations from the ANN field, creating new INFO fields with prefix 'snpeff_' for each annotation. This calculation takes the ANN field from snpEff annotations, which may contain multiple annotations for a single variant, and explodes it so that each annotation is represented in separate INFO fields with a 'snpeff_' prefix. This allows for easier access to individual annotations and facilitates downstream analysis.

1.18 SNPEFF__ANN__EXPLODE__JSON

Explode snpEff annotations in JSON format

Explode snpEff annotations from the ANN field, creating a new INFO field 'snpeff_json' in JSON format that contains all annotations. This calculation takes the ANN field from snpEff annotations, which may contain multiple annotations for a single variant, and explodes it into a structured JSON format stored in a new INFO field named 'snpeff_json'. This allows for easier access to all annotations in a single field and facilitates downstream analysis.

1.19 SNPEFF__ANN__EXPLODE__UNIQUIFY

Explode snpEff annotations with uniquify values

Explode snpEff annotations from the ANN field, creating new INFO fields with prefix 'snpeff_uniquify_' for each annotation and ensuring unique values. This calculation takes the ANN field from snpEff annotations, which may contain multiple annotations for a single variant, and explodes it so that each annotation is represented in separate INFO fields with a 'snpeff_uniquify_' prefix, ensuring that only unique values are retained. This allows for easier access to individual annotations and facilitates downstream analysis.

1.20 SNPEFF__EXTRACT

HGVS nomenclatures from snpEff annotation

Extract HGVS nomenclatures from snpEff annotation (field ANN) and create new INFO fields with prefix 'snpeff_' (e.g. snpeff_hgvs, snpeff_impact, snpeff_gene_name...). This calculation parses the ANN field from snpEff annotations, extracting relevant information such as HGVS nomenclatures, impact, gene name, etc., and creates new INFO fields with a 'snpeff_' prefix for easier access and downstream analysis.

1.21 SNPEFF_HGVS

HGVS nomenclatures from snpEff annotation

Extract HGVS nomenclatures from snpEff annotation (field ANN) and create new INFO field 'snpeff_hgvs'. This calculation specifically targets the extraction of HGVS nomenclatures from the ANN field of snpEff annotations, creating a new INFO field named 'snpeff_hgvs' that contains the extracted HGVS information for easier access and downstream analysis.

1.22 TRANSCRIPTS_ANNOTATIONS

Perform transcripts annotations and generate a transcripts table/view (using JSON parameters file)

Perform transcripts annotations and generate a transcripts table/view. This calculation allows for the inclusion of transcript annotations in both JSON and structured formats, providing flexibility in how transcript information is stored and accessed within the variant analysis pipeline. The specific format(s) used can be determined based on parameters specified in the JSON parameter file in 'transcripts' section (see help.parameters.pdf), or directly in the calculation parameters.

1.23 TRANSCRIPTS_JSON

Perform transcripts annotations and export into INFO field in JSON format (field 'transcripts_json')

Perform transcripts annotations and generate a transcripts table/view. This calculation allows for the inclusion of transcript annotations in both JSON and structured formats, providing flexibility in how transcript information is stored and accessed within the variant analysis pipeline. The specific format(s) used can be determined based on parameters specified in the JSON parameter file in 'transcripts' section (see help.parameters.pdf). Then and add transcripts fields into INFO fields in structured format (field 'transcripts_ann'). This calculation allows for the inclusion of transcript annotations in a JSON format, facilitating the integration of transcript information into the variant analysis pipeline.

1.24 TRANSCRIPTS_ANN

Perform transcripts annotations and export into INFO field in structured format (field 'transcripts_ann')

Perform transcripts annotations and generate a transcripts table/view. This calculation allows for the inclusion of transcript annotations in both JSON and structured formats, providing flexibility in how transcript information is stored and accessed within the variant analysis pipeline. The specific format(s) used can be determined based on parameters specified in the JSON parameter file in 'transcripts' section (see help.parameters.pdf). Then and add transcripts fields into INFO fields in structured format (field 'transcripts_ann'). This calculation allows for the inclusion of transcript annotations in a structured format, facilitating the integration of transcript information into the variant analysis pipeline.

1.25 TRANSCRIPTS_EXPORT

Export transcripts table/view as a file (using JSON parameters file)

Export transcripts table/view as a file. This calculation allows for the export of the transcripts table or view generated from transcript annotations to an external file format such as TSV or CSV. The parameters for this calculation can be specified in the JSON parameter file, either in 'transcripts' section or directly in the calculation parameters (see help.parameters.pdf), including options for the output file path, format, and any filters to apply during export.

1.26 TRANSCRIPTS_PRIORITIZATION

Prioritize transcripts with a prioritization profile (using JSON parameters file)

Prioritize transcripts with a prioritization profile. This calculation allows for the prioritization of transcripts based on a predefined profile specified in the JSON parameter file, either in 'transcripts' section or directly in the calculation parameters (see help.parameters.pdf), helping to identify the most relevant transcripts for further analysis.

1.27 TRANSCRIPTS_PRIORITIZATION_STRICT

Prioritize transcripts with a prioritization profile (using JSON parameters file)

Prioritize transcripts with a prioritization profile, ensuring that all needed annotations are present and considered. This calculation allows for the prioritization of transcripts based on a predefined profile specified in the JSON parameter file, either in 'transcripts' section or directly in the calculation parameters (see help.parameters.pdf), helping to identify the most relevant transcripts for further analysis.

1.28 TRIO

Inheritance for a trio family

Inheritance for a trio family. This calculation assesses the inheritance pattern of a variant within a trio family pedigree (father, mother, and child). It helps in understanding the transmission of genetic variants and identifying potential de novo mutations. Options for this calculation can be specified in the JSON parameter file, directly in the calculation parameters (see help.parameters.pdf). The parameter 'trio_samples' allows for the specification of the sample names for father, mother, and child:

```
{"father": "sample_father", "mother": "sample_mother", "child": "sample_child"}
```

This parameter can also be specified in a JSON file (containing the trio sample names):

```
"/path/to/trio_samples.json"
```

If no pedigree information is provided, the calculation will use the first 3 samples in the VCF file.

1.29 VAF

Variant Allele Frequency (VAF) harmonization

Variant Allele Frequency (VAF) harmonization. This calculation normalizes the Variant Allele Frequency (VAF) across different samples and callers to ensure consistency in VAF representation. It helps in comparing VAF values across samples and callers by applying a harmonization process.

1.30 VAF_STATS

Variant Allele Frequency (VAF) statistics

Variant Allele Frequency (VAF) statistics. This calculation provides statistical measures for the Variant Allele Frequency (VAF) across different samples. It helps in understanding the distribution and variability of VAF values.

1.31 VARIANT_CHR_POS_ALT_REF

Create a variant ID with chromosome, position, alt and ref

This calculation generates a variant ID with chromosome, position, alt and ref, with format '#CHROM_POS_REF_ALT'. Useful for variant identification and comparison across datasets

1.32 VARIANT_FILTER

Filter variants based on specified criteria (using SQL parameters and sample list)

Filter variants based on specified criteria. This calculation allows for the filtering of variants using SQL-like parameters specified in the JSON parameter file in the calculation parameters (see help.parameters.pdf), including options for the filtering conditions and any additional criteria (such as a list of samples) to apply during the filtering process. Example that filter variants on ClinVar pathogenic, on DP >= 100 for 'sample1', selection of only 2 samples, and ensure only not null genotypes:

```
{
  "filter_name": "Filter on ClinVar pathogenic, DP for sample1, select only 2 samples, and ensure only not null",
  "where_clause": "INFOS.CLNSIG = 'pathogenic' AND SAMPLES.sample1.DP >= 100",
  "sample_list": ["sample1", "sample2"],
  "genotype_filter": true
}
```

1.33 VARIANT_ID

Variant ID generated from variant position and type

Variant ID generated from variant position and variation (ref and alt) and type (SVTYPE). This calculation creates a unique identifier for each variant, facilitating variant tracking and comparison across datasets. Options for this calculation can be specified in the JSON parameter file, directly in the calculation parameters (see help.parameters.pdf). The parameter 'variant_id_tag' allows for the specification of the INFO tag to be used for the variant ID. By default, it uses 'variant_id',

but it can be changed to any other INFO tag name as needed. For example, to use 'varid' as the INFO tag for the variant ID, you can specify:

```
{ "variant_id_tag": "varid" }
```

Other options as 'variant_id_tag_info' allows for the specification of the description in the VCF header, and 'keep_variant_id_tag_column' allows for keeping the variant ID tag column in the 'variants' table for further analysis.

As an example of the JSON parameter file, you can specify:

```
{ "variant_id_tag": "varid", "variant_id_tag_info": "Variant ID generated from variant position and type", "k
```

1.34 VARIANT__ID__VARID

Variant ID generated from variant position and type, using 'varid' as INFO tag

Variant ID generated from variant position and variation (ref and alt) and type (SVTYPE). This calculation creates a unique identifier for each variant, facilitating variant tracking and comparison across datasets.

1.35 VARTYPE

Variant type (e.g. SNV, INDEL, MNV, BND...)

Determine the type of variant based on its characteristics, such as the length of the reference and alternate alleles, and the presence of structural variant information. This calculation classifies variants into types like SNV, INDEL, MNV, BND, etc., which is essential for downstream analysis and interpretation.

1.36 VEP__EXTRACT

HGVS nomenclatures from VEP annotation

Extract HGVS nomenclatures from VEP annotation (field CSQ) and create new INFO fields with prefix 'vep__' (e.g. vep_hgvs, vep_impact, vep_gene_name...). This calculation parses the CSQ field from VEP annotations, extracting relevant information such as HGVS nomenclatures, impact, gene name, etc., and creates new INFO fields with a 'vep__' prefix for easier access and downstream analysis.

1.37 VEP__EXTRACT__REFSEQ

HGVS nomenclatures from VEP annotation using RefSeq transcripts

Extract HGVS nomenclatures from VEP annotation (field CSQ) and create new INFO fields with prefix 'vep__' (e.g. vep_hgvs, vep_impact, vep_gene_name...). This calculation parses the CSQ field from VEP annotations, extracting relevant information such as HGVS nomenclatures (with refSeq), impact, gene name, etc., and creates new INFO fields with a 'vep__' prefix for easier access and downstream analysis.